Summary of Changes - resubmission of “Erasure-Compliant, Differentially Private Distinct Counting under Continual Observation”

Prepared for resubmission (PoPETs 2027.2 or later, per the 2026.4 decision). This document maps every reviewer concern from the 2026.4 reviews (45A-45D) and the meta-review to the change made. Section numbers refer to the revised paper.

Delta for the PoPETs 2027 issue 3 submission (template, mandatory sections, figures)

This section lists the changes made after the strengthener revision, to prepare the manuscript for the 2027 submission template. No reviewer-facing claim in the sections below was weakened.

- 2027 template. The manuscript now builds on the PoPETs 2027 submission template files (`acmart.cls`, `popets.sty`, `ACM-Reference-Format.bst`, `acmnumeric.bbx/.cbx`). Two non-template formatting overrides were removed: the `balance=false` class option and a `\acmPrice` redefinition; the unused `totpages` package and a forced `\newpage` before the appendix were also removed.
- Mandatory sections. Added Ethical Considerations, Open Science and AI Use, in the order required by the template (after acknowledgements, before references). The generative-AI sentence that previously sat in the acknowledgements moved, verbatim, to AI Use.
- Figures. The four evaluation figures were missing from the repository and could not be reused: the earlier versions were drawn from hard-coded numbers that the revised paper had already retired (for example the 202 ms MAU latency and 51.9 s ingest at 1M users). All four are now regenerated by `dp-dau-mau/eval/make_paper_figures.py` from committed result files and committed code only. The latency figure now shows the fresh and cached MAU numbers that Table “Measured Scalability” reports.
- Correction of the RDP saving. The budget figure is now computed with the repository’s own accountant. At 30 daily MAU releases with epsilon = 0.5 and delta = 1e-6 it gives epsilon_total of about 3.04 against 15.0 for naive summation (about 80% lower), and about 11.1 against 182.5 at 365 releases (about 16.4x), which matches the strengthener results. The earlier text claimed about 25% (30 releases) and about 50% (365 releases); those numbers were not reproducible from the code and were replaced in Sections 2.2, 5 and 7.7, the figure caption and description, the telemetry case study, and the proof of the lemma in the appendix.

Delta since 2026.4: measured results + two strengtheners (**`popets-strengtheners`** branch)

This section documents the changes made on top of the 2026.4 revision described below. The 2026.4 revision fixed the paper’s formal content (definitions, theorems, threat model). This delta fixes the paper’s performance evaluation so that every quantitative claim is reproducible from the code actually committed to this branch, using the measured numbers in `ROARING_BENCHMARK.md` and `STRENGTHENERS_RESULTS.md` (both in the repo root). Framing: honest cold-vs-cached reporting, never restoring an old claim the measurements don’t support. Where a number changed, we state the old claim and the measured replacement explicitly.

What shipped on this branch (previously missing/orphaned)

- A real Roaring Bitmap backend (`RoaringSketch`, `src/dp_core/sketches/roaring_impl.py`), hashing to 64 bits (personalized BLAKE2b), wired into the pipeline’s backend factory. Prior to this branch, the paper described a Roaring backend and cited 32-bit hashing, but no Roaring implementation was committed to any branch, and the code that did exist elsewhere (uncommitted, orphaned) used a lossy 32-bit truncation that collides distinct users (measured: on the order of 100 colliding pairs at 1M users). §6 (Implementation) is corrected to describe the 64-bit design that ships, and to explain why 32-bit was rejected rather than silently kept as the description.
- A fast, append-only binary-log ingest engine (`src/dp_core/storage/`), replacing the previously-committed per-event SQLite ledger as the pipeline’s ingest path.
- A version-fingerprinted window-union cache (`WindowManager.get_mau`, `src/dp_core/windows.py`): memoizes the exact 30-day union for repeat MAU queries of an unchanged window, invalidated per-day on ingest or erasure. This is the pre-noise analogue of the paper’s existing “Noise Caching with Versioning” mechanism (§4.5), and is now documented as its own paragraph, “Window-Union Caching with Versioning (Pre-Noise).”

- All 56 tests green on this branch; `ROARING_BENCHMARK.md` and `STRENGTHENERS_RESULTS.md` document methodology and full measured tables.

Claim-by-claim: old (unreproducible) → measured (this branch)

#	Old claim (paper, pre-strengtheners)	Measured, this branch	Where fixed
1	115,000 events/sec ingest	≈107,817 events/s at 1M users (114,427/s at 10k) - matches within ~6%, now reproducible on the committed binary-log engine	Abstract, Intro contributions, §6 (Implementation), §7.8 (Performance)
2	42× ingest speedup vs. an unmeasured "naive Python" baseline (2,700 events/s)	≈11.75× vs. the previously-committed, measured per-event SQLite ledger (9,176 events/s → 107,817 events/s); the 2,700/s baseline was never re-measured on this branch, so the 42× figure is retired rather than restated	§7.8 (Performance), Table "Ingest Throughput & Peak Memory"
3	202 ms MAU query latency (p99)	Dropped entirely. It came from an orphaned, uncommitted, lossy 32-bit-truncated backend variant that silently collides users - not exact, and not what ships. Replaced with two honest numbers: fresh/first query of a 30-day window ≈2.7 s p50 / ≈4.5 s p99 at 1M users (the real cost of an exact 64-bit Roaring union of 30 daily bitmaps), and cached/repeat query of the same window ≈0.57 ms p50 / ≈1.45 ms p99 at every scale tested (10k-1M), via the new version-fingerprinted window-union cache - exact, not approximate	Abstract, Intro contributions, §4.3 (State Backend), §4.5 (new "Window-Union Caching with Versioning" paragraph), §7.8 (Performance, with a dedicated "why we no longer report a single MAU latency number" paragraph)

#	Old claim (paper, pre-strengtheners)	Measured, this branch	Where fixed
4	223× MAU speedup vs. naive baseline (45.2 s → 0.202 s)	Retired. It was computed against the unreproducible 202 ms figure. Honest fresh-query comparison against the same naive 45.2 s baseline is only ≈10×; a cached repeat query is ≈31,000× faster than that baseline, reported but flagged as not apples-to-apples (the naive baseline never had the option to cache a repeat query either)	§7.8 (Performance)
5	~1.15 GB memory footprint at 1M users	≈1.55-1.64 GB whole-process on the fast binary-log path (measured). The old 1.15 GB figure matches the previously-committed SQLite-ledger path (measured here at 1,215.6 MB), not the fast path the evaluation now uses - this is stated explicitly rather than silently carried forward under the new engine's name	§7.8 (Performance), §9 (Discussion, Scalability Considerations)
6	(not previously reported) Roaring vs. plain-`set` memory comparison	New, measured: Roaring's serialized state is ≈1.59× smaller than a plain Python `set` (22.0 vs. 35.0 bytes/user, stable across scale), and process RSS at 6M user-days is ≈33% lower (526.5 MB vs. 784.9 MB). Also disclosed: on a genuinely new window, `set`'s C-level hash-set union is actually faster than Roaring's sparse 64-bit union (778 ms vs. 2,821 ms p50 at 1M) - Roaring's advantage is memory, not fresh-query speed, at this exact/64-bit configuration	§7.8 (Performance), new Table "Set vs. Roaring"

#	Old claim (paper, pre-strengtheners)	Measured, this branch	Where fixed
7	32-bit user-ID hashing described in §6 (Implementation)	Corrected to 64-bit (personalized BLAKE2b), matching the committed `RoaringSketch`. Prose now explains why 64-bit was chosen (exactness) and its performance consequence (sparse, non-SIMD-friendly containers, hence the fresh-query union cost above)	§6 (Implementation)

Two strengtheners added to the evaluation

1. History-independence exhibit (new §7.3, "History-Independence Exhibit," forward-referenced from Definition "History-Independent Deletion" in §3.4): empirically tests the paper's own conditional definition against the repository's actual backend code, rather than leaving it asserted. Result: the exact `set` backend and the committed exact Roaring backend match the rebuild-from-D{u} state in 100% of 2,460 trials each, across every regime tested (including production k=4096); the shipped `kmv` sketch matches unconditionally only when n ≤ k, and diverges in 23-35% of trials once bottom-k truncation occurs (mean cardinality error 10.4-3,725, worst case up to 16,370 at production k). This is a genuine, reproducible confirmation of the paper's own conditional claim, caught empirically rather than assumed. A Theta backend is named in the reference architecture but does not exist in the repository, committed or otherwise; reported as untested (N/A), not simulated.
2. Tree-aggregation comparison (Future Work, "Tree Aggregation for Smoother Releases," and Limitations): quantifies the potential benefit of tree aggregation using the repo's own `PrivacyAccountant._best_from_curve` on a synthesized 365-release RDP curve, rather than hand-derived formulas alone. Finding is asymmetric, reported honestly rather than as a uniform win: for DAU (Laplace, δ=0, naive summation only), tree aggregation could reduce the 365-day cumulative budget from ε≈109.5 to ε≈9.49 for the same per-release accuracy (≈11.5×; confirmed by an 800-trial Monte Carlo simulation on a disclosed synthetic additive proxy, measuring 14-35× error reduction at a matched budget). For MAU (Gaussian), the shipped accountant's existing RDP composition already achieves ≈16.4× over naive summation (ε≈11.1 vs. ≈182.5) independent of tree aggregation; layering a 10-level tree on top of that same budget adds a smaller, real gain (up to ≈6× per-level accuracy for low-tree-depth releases). This asymmetry - large win where the shipped mechanism only has naive composition (DAU), smaller-but-real win where it already composes tightly (MAU) - is reported as the honest takeaway rather than a single flattering number.

Not changed

- The 32-bit-vs-64-bit and cold-vs-cached corrections above do not touch the paper's formal content (Definitions, Theorem, Lemma, Propositions, proofs) from the 2026.4 revision, which remains as described below.
- DAU/MAU accuracy-vs-ε results (Table "Accuracy metrics") are unaffected - the exact backend's accuracy story was never based on the retracted 32-bit or 202 ms figures.
- The adversarial "toggle storm" latency figure (<210 ms for MAU, §7.5) describes a 500-user workload, not the 1M-user scale corrected above, and was not re-measured as part of this delta; it is left as-is.

Meta-review critical weaknesses

(1) "Sections 3-6 too high-level; insufficient detail on how techniques combine." - Merged the two redundant threat-model sections (old §3.2 + old §4) into one precise §3.2 with explicit trust semantics, adversary capabilities, the KMS contract, gateway metadata visibility, and the guarantees summary. - Removed a stray implementation/latency paragraph from old §3.3 (it belonged in the evaluation) and rewrote the flippancy connection. - The

design sections now state mechanisms concretely: the cross-epoch overlap protocol is specified operationally and proven correct (§5.2, Prop. “Cross-Epoch Window Correctness”); the tombstone lifecycle now distinguishes eager hot-state update, historical replay via compaction, settlement, and purge (§5.3); backend admission is tied to a formal property (§5.4). - Introduction rewritten to set up the problem and the tension (continual observation vs. retroactive erasure) with intuition before mechanisms; contributions reformatted consistently (fixing the malformed item 5).

(2) “Lack of formalization of the actual protection (what erasure-compliance means; what GDPR allows/not).” - New Definition (History-Independent Deletion) (§3.4): the backend property required for the erasure guarantee to hold; text explains why append-only sketches fail it, under what conditions KMV/Theta satisfy it, and that exact sets satisfy it unconditionally. Backend admission requires the property (§5.4). [Raised by 45A(C) and 45A’s post-rebuttal note.] - Existing Definition (Post-Erasure Release Semantics) is now connected to the new property and to the deletion lifecycle. - New Theorem (Deployment-Horizon Guarantee) (§6) with proof (App. A): the ledger-enforced (ε_tot, δ_tot) bound over ALL releases, including deletion-triggered re-releases, answering “over how long are the theorems valid” [45D]. Per-release propositions are explicitly scoped as per-release. - Lemma (RDP Composition Benefit) now has a proof (App. A) [45D: “I do not see a proof for lemma 6”]. - GDPR formal positioning: pseudonymization vs. anonymization under Recital 26 was already stated in §2.1; the CCTV case study’s contradictory “anonymized at the edge” sentence is corrected to pseudonymized with the Recital-26 consequence spelled out [45D]. Why tombstones are compatible with Article 17 (transient control-plane artifacts; hashed only; purged at settlement; end state equivalent to never-ingested) is now argued explicitly in §5.3 [45D: “why is it okay to keep deletion records?”]. - Whether erasure requests themselves are DP-protected is now answered explicitly in the threat model (§3.2): request fact/timing are control-plane data visible to the aggregation tier; DP protects presence in all released statistics [45A other-comment].

(3) “Concerns about relevance and scalability of the implementation and case study.” - New primary case study: product telemetry (§9.1), grounded in the exact configuration the evaluation measures; CCTV demoted to a stress variant (§9.2) with its purpose stated (testing the architecture where the identity layer is weakest) [45C, and 45C’s rebuttal-response question]. - Simulation methodology stated up front for both case studies (synthetic traces from the evaluation generators; real pipeline; no fielded-deployment claims); “lessons learned” reframed as “design considerations surfaced by the exercise,” fixing the simulated-vs-real mismatch [45B]. - New scalability honesty in Discussion: exact bitmaps are explicitly scoped to single-gateway scale; sketch backends are the intended distributed configuration; open engineering questions (serialization, deletion fan-out, distributed accounting) named; claims scoped to the measured single-node profile [45A evaluation comment].

Review 45A main comments

- (A) Salt rotation vs. sliding windows (W=62 across a boundary): resolved by the formalized overlap protocol in §5.2 - dual-salt bridge table during the first W_max−1 days of each epoch, with a correctness proposition (proof sketch inline) and erasure coverage of the bridge state. Epoch length must exceed W_max; W_max default raised to 62 in the exposition to match the reviewer’s scenario.
- (B) KMV/Theta vs. Roaring confusion: already-clarified in the submission’s §2.3 and §7.2 (“Backend Clarification”); now additionally tied to the formal admission property in §5.4, and the abstract/intro no longer imply a sketch is deployed in the evaluation.
- (C) Erasure never formalized / sketches would not work: the reviewer is correct, and the new Definition (History-Independent Deletion) adopts exactly the formalization they suggested (state after deletion ≡ state built from D\{u}); KMV/Theta are stated to qualify only under conditions.
- (D) What the KMS hides and from whom: new “KMS contract” paragraph in §3.2 (secrets, interface, who is blinded, consequence of compromise, and the fact DP survives salt compromise).
- Jain et al. lower bound misquote: corrected throughout to $\Omega(\min(T^{\{1/3\}/\varepsilon}\{2/3\}, T))$ (was already fixed in the current text; re-verified at all three occurrences), and the implication of their matching upper bound (correlated noise can beat per-release noise) is now acknowledged in §3.3, Limitations, and Future Work.

Review 45B

- Data-minimization question (sketch vs. bitmap attack surface under gateway compromise): new dedicated paragraph in §5.4 quantifying the disclosure difference and exposing it as a deployment-time choice.

- Tombstones and Roaring bitmaps defined at first use (tombstones at first mention in §3.4; Roaring definition at first mention in §2.3).
- backend.remove on historical backends: now explicit as the "Tombstone Replay" step with a settlement condition (§5.3).
- Trivial theorems: per-release results are stated as propositions with one-line framing of why they are stated at all (compositional inputs to the horizon theorem); duplicate inline proofs removed (proofs live in App. A only).
- Case-study simulation procedure clarified (§9 preamble + per-study methodology).

Review 45C

- Reorganized §3-6 (merge of threat-model sections; displaced content moved; tighter flow); intro expanded with motivation and intuition before mechanisms.
- Implementation/evaluation share of the paper rebalanced by strengthening the formal core (new definition, theorem, proposition, proofs) rather than deleting measured content.
- CCTV case study replaced as primary by telemetry (see meta-(3)).

Review 45D

- Trusted vs. honest-but-curious contradiction resolved with precise wording (§3.2).
- GDPR-after-pseudonymization corrected in the case study (§9.2).
- Gateway seeing which edge device sent which key: acknowledged explicitly (§3.2, "What the gateway sees"); mixing declared out of scope rather than implied.
- Fig. 2 caption now states the timeline is schematic and not to scale.
- Deletion-record retention justified (§5.3, "Why tombstones are compatible with erasure").
- Theorem validity over time: Deployment-Horizon theorem (§6); Lemma proof added.
- §7.1: salt-derivation memoization location was already stated (gateway ingestion layer, keyed by epoch, invalidated on rotation); retained.
- "82% is impressive" removed; coverage now described by where it concentrates (privacy-critical paths) including a property-based test of history independence.
- §8.2: new "What drives accuracy" paragraph identifying exactly which variables affect error (ε, true count; not deletion rate, not window length) and why.
- §8.3: naive baseline justified as a quantification of non-compliance error, not a competitive alternative.
- §8.4: churn rate of the storm already stated (≈99% effective churn; 500 users × up to 5 flips); retained.
- Editorial: first paragraph rewritten; contributions list reformatted (broken item 5 fixed); figure captions annotated. [A full grammar pass was applied to the sections rewritten above; remaining sections were already edited post-rebuttal.]

Not changed (and why)

- The evaluation's exact-backend instantiation is retained (with the clarified rationale): isolating DP noise from sketch error remains the right evaluation design; the minimization trade-off and distributed-scale positioning are now stated instead of implied.
- Tree aggregation (correlated noise) remains future work; the revision now acknowledges explicitly that it can asymptotically beat per-release noise, per the corrected reading of Jain et al.